



Institución:

Instituto Tecnológico de Tepic

Curso:

Programación Web

Unidad 4:

JavaScript

Título:

Investigación

Autor:

Isais Nava Gandhi Antonio – 234005666

Docente:

Irving Yair Nava Hernández

Fecha:

13 – 07 – 2026

INDICE

Características Básicas del Lenguaje	3
Estructura de Control	3
Condicionales.	3
Bucles.	3
Tipos de Datos	3
Primitivos.....	3
Referencias.	4
Funciones	4
Declaradas.....	4
Expresadas.....	4
Funciones Flecha (Arrow Functions).	4
Objetos y Arrays	4
Objetos.	4
Arrays.....	4
Eventos y Manipulación del DOM	5
DOM (Document Object Model).	5
Eventos.	5
Asincronía.....	5
Callbacks.....	5
Promesas.....	5
Async / Await.....	5



JavaScript

Características Básicas del Lenguaje

Interpretado: Esto significa que el navegador lee tu código y lo ejecuta línea por línea en el momento. No necesitas un programa especial que “compile” o transforme el código en otra cosa antes de poder ver si funciona.

Dinámico: Aquí las variables no son celosas con su tipo de dato. Puedes crear una variable que guarde un número y, un poco más adelante, meterle un texto. El lenguaje no se va a quejar ni a romper por eso.

Multiparadigma: No te obliga a programar de una sola forma. Si te gusta usar funciones sueltas, puedes hacerlo; si prefieres trabajar con objetos, también; o puedes mezclar estilos según lo que mejor se adapte a tu proyecto.

Estructura de Control

Condicionales.

If/else: Es el clásico “si pasa esto, haz aquello”. Evalúa si algo es verdadero para ejecutar, un bloque de código. Si no se cumple, puede saltar a otra opción con el else.

Switch: Es ideal cuando tienes muchas opciones para una sola variable. Funciona como un menú donde el programa busca el caso exacto que coincida con tu variable.

Bucles.

For y while: Sirven para repetir un bloque de código varias veces. Se siguen ejecutando mientras la condición que se le diste siga siendo verdadera.

Tipos de Datos

Primitivos.

Son datos simples y básicos. Cuando los pasas a otra variable, se saca una copia idéntica e independiente. Son los siguientes:

- **String:** Textos (Siempre van entre comillas).
- **Number:** Números (tanto enteros como decimales).
- **Boolean:** Solo pueden ser true o false.
- **Undefined:** Significa que la variable existe, pero nadie le ha puesto un valor todavía.
- **Null:** Es cuando dices a propósito que una variable esta vacía.

- Symbol y BigInt: Se usan para cosas muy específicas, como identificadores únicos o números gigantescos.

Referencias.

Son datos complejos. Aquí las variables no guardan el valor real, sino una “dirección” de memoria que apunta a donde esta la información. Si duplicas la variable, ambas apuntan al mismo lugar.

- Object: Colecciones de datos
- Array: Listas de cosas
- Function: Bloques de código que hacen tareas

Funciones

Las funciones son bloques de código que guardas bajo un nombre para poder usarlos (llamarlos) las veces que quieras sin tener que volver a escribir todo.

Declaradas.

Usan la palabra clave function. Tienen un “Superpoder” llamado hoisting, que hace que el navegador las detecte antes de que corra el código, así que puedes usarlas incluso en líneas de arriba antes de donde las escribiste.

Expresadas.

Es cuando creas una función y la guardas dentro de una variable (const o let). Con estas no puedes hacer trampa: solo las puedes usar después de la línea donde las creaste.

Funciones Flecha (Arrow Functions).

Son las más modernas y usan el símbolo =>. Son geniales porque ahorran mucho espacio al escribir y además resuelven dolores de cabeza con una palabra clave muy confusa del lenguaje llamada this.

Objetos y Arrays

Son las estructuras que usamos para guardar y ordenar mucha información junta.

Objetos.

Imagínalos como la ficha técnica de algo. Guardan datos en parejas de "clave y valor" (por ejemplo: color: "rojo", puertas: 4). Sus características se llaman propiedades y sus acciones se llaman métodos.

Arrays.

Son simplemente listas ordenadas de cosas. Cada elemento tiene un número de posición llamado índice (y ojo, siempre se empieza a contar desde el 0). Tienen herramientas integradas muy útiles como .map() (para transformar la lista) o .filter() (para quedarte solo con lo que te interesa).

Eventos y Manipulación del DOM

Esto es lo que hace que el usuario realmente interactúe con la página.

DOM (Document Object Model).

Cuando el navegador abre una página HTML, la traduce a un mapa de objetos que JavaScript puede entender. Gracias al DOM, JavaScript puede entrar a la página y cambiar un texto, quitar un botón o cambiar un color a mitad de camino.

Eventos.

Son las cosas que pasan en la página, sobre todo cuando el usuario interactúa (dar un clic, escribir algo, mover el ratón). JavaScript se queda "escuchando" estos eventos con algo llamado `addEventListener` y activa una función cuando ocurren.

Asincronía

JavaScript solo puede hacer una cosa a la vez. Si le pides algo que tarda mucho (como traer información desde internet), la página se congelaría. Para evitar esto, se usa la asincronía, que ha evolucionado así:

Callbacks.

La forma antigua. Le pasas una función a otra para decirle: "Oye, haz esta tarea pesada y, cuando termines, avísame ejecutando esta función". Si encadenas muchas, el código se vuelve un laberinto ilegible.

Promesas.

Una forma más limpia. Es un objeto que te promete que te dará una respuesta en el futuro (ya sea que todo salió bien o que hubo un error). Se manejan con `.then()` y `.catch()`.

Async / Await.

Es la manera moderna y la que todo el mundo usa hoy. Es un truco visual que hace que el código asíncrono se lea de corrido, igual que el código normal. Hace que todo sea mucho más fácil de entender y evita que te equivoques.